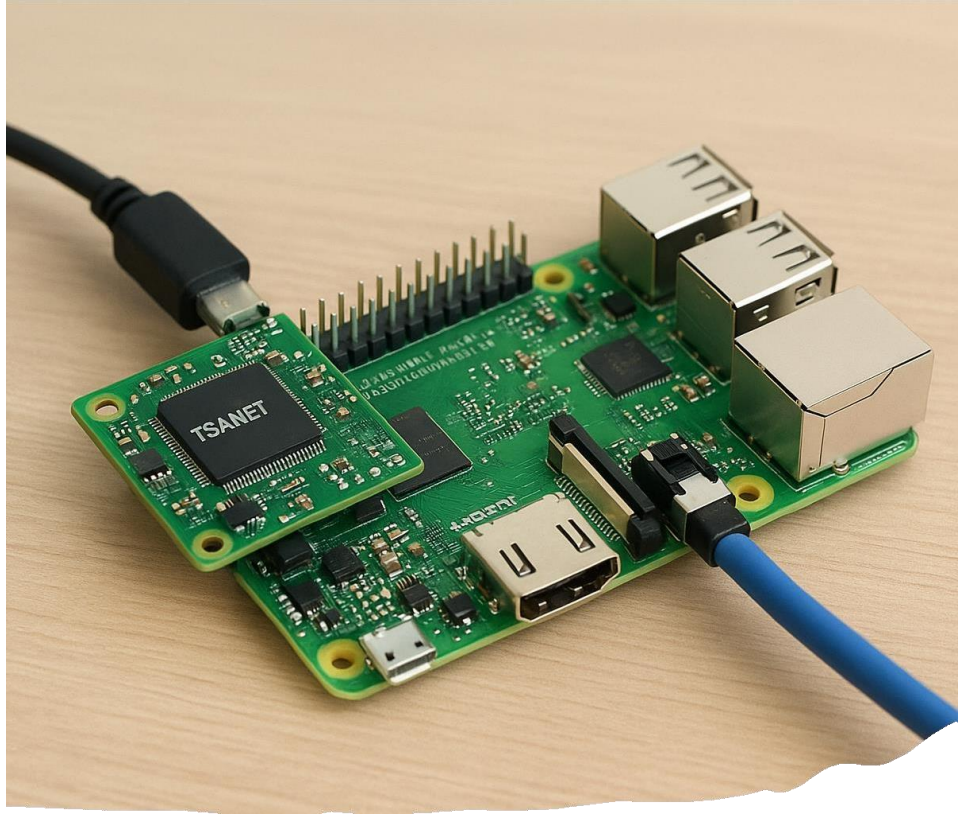


Lightweight Real-Time Intrusion Detection for IoT Applications Using TSANet Deployed on Raspberry Pi with Mininet Traffic Simulation

CONTRIBUTOR: ROHAN KULKARNI
02FE22BCS083
KLE TECH UNIVERSITY

MENTOR: DR.RYAN
DIAS DEPT OF CS
KLE TECH UNIVERSITY

PROFILE

Introduction

- Increasing frequency and sophistication of cyberattacks (DoS, DDoS, brute force, Heartbleed etc.).
- Traditional Intrusion Detection Systems (IDS) struggle with scalability and real-time detection.
- Deep learning offers advanced feature extraction and accurate classification of malicious traffic.
- Deploying IDS on edge devices (e.g., Raspberry Pi) enables local, real-time detection close to the traffic source through mininet

Empathy

- **Problem for users:** Difficulty confirming whether a deployed model actually detects attacks in real network-like conditions.
- **Our solution:** A deployed TorchScript model on Raspberry Pi that monitors flows produced by Mininet and signals detection via a visible LED blink.

BACKGROUNDC MOTIVATION

Intrusion Detection Systems (IDS) are essential for identifying malicious network activities. Traditional IDS solutions are usually dataset-based and tested offline. Real-time evaluation often requires expensive hardware or complex testbeds. Deep learning–based IDS has shown superior detection accuracy on benchmark datasets (e.g., CICIDS-2017).

Need for a practical, low-cost testbed to evaluate IDS models beyond static datasets. Raspberry Pi offers a lightweight, real-time deployment platform to validate model inference. Mininet provides a controlled, reproducible environment to generate both benign and malicious traffic. LED-based alerting ensures simple and clear demonstration of real-time attack detection.

LITERATURE REVIEW

1. Paper Title : A Lightweight Optimized Deep Learning-based Host-Intrusion Detection System Deployed on the Edge for IoT

- CNN-based approach with 5-layer architecture (Conv1D, MaxPooling1D, Flatten, Output). **Edge deployment** on Raspberry Pi and microcontrollers. **Model optimization** through weights pruning, post-training quantization, and weight clustering. **TensorFlow Lite** conversion for lightweight deployment. Training: 10 epochs, batch size 32, parameters reduced from 26,365 to 645.
- **MQTT-IOT-IDS2020 dataset** (uniflow features version). Generated from simulated MQTT network with 12 sensors, broker, camera, and attacker. **5 labels:** Normal traffic, Aggressive scan, UDP scan, Sparta SSH brute-force, MQTT brute-force attack. Data split: 60% training, 20% validation, 20% testing.

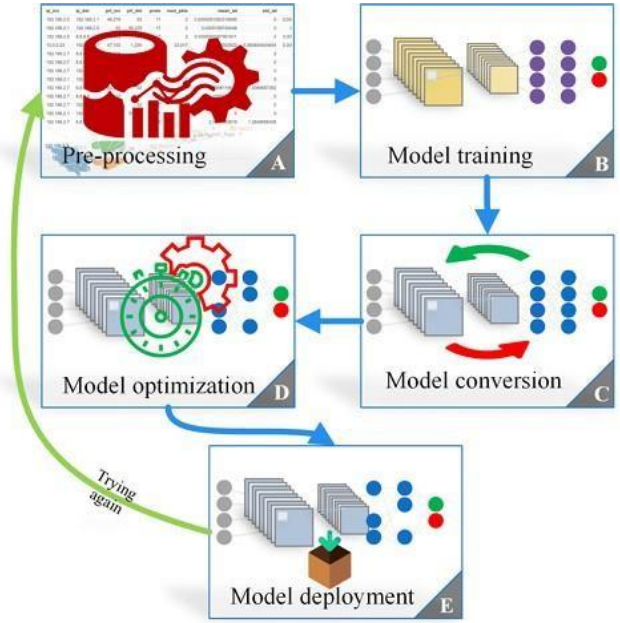


Figure 4. DL-IDS preparation and deployment steps

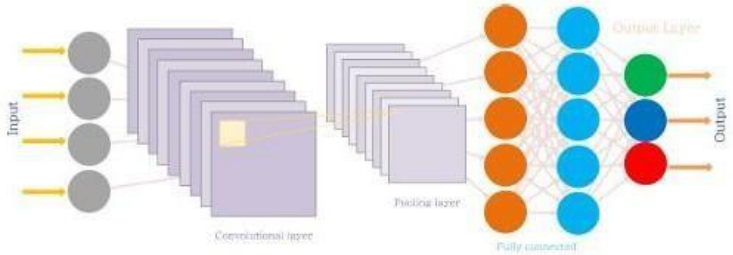


Figure 1. CNN architecture

- **Accuracy:** 96.69% (optimized model), 99.74% (original model). **Model size:** Reduced from 343 KB to 2704bytes. **Memory usage:** Significantly reduced for edge deployment. **Inference time:** Faster due to edgeprocessing.
- **Generalization challenge:** Need for device-specific customization rather thanuniversal solution. **Resource constraints:** Limited by IoT device computational capabilities.
- **Scalability:** Difficulty in standardizing across diverse IoT hardware.

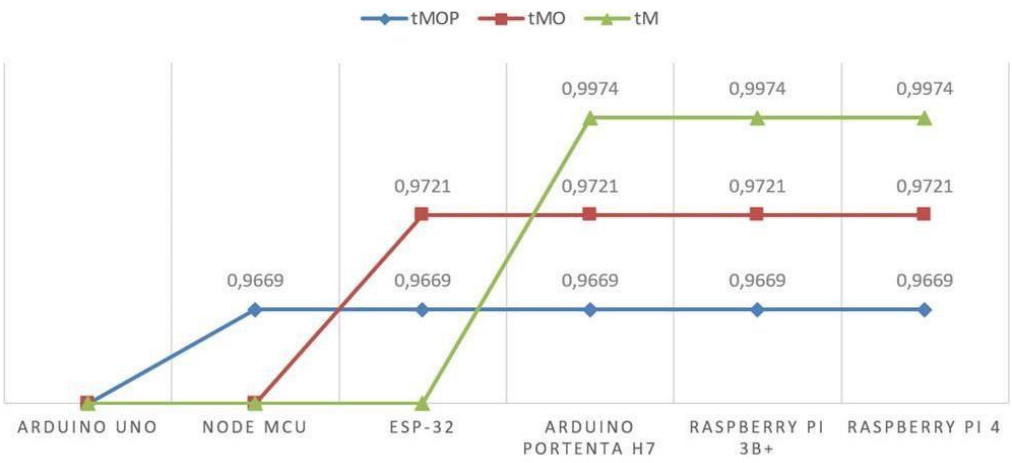


Figure 7. The suited model for each device

LITERATURE REVIEW

2. Paper Title : Anomaly-based Intrusion Detection System for IoT Networks through Deep Learning Model

- **CNN-based anomaly detection model.** Sequential CNN with categorical/binary cross-entropy loss, Adam optimizer, 0.5 dropout, 0.2 test size, batch size 10, trained for 30 epochs. **Deep learning** for traffic analysis to classify normal vs abnormal behavior. **Parameter sharing** and sparse connections used for scalability.
- **NID dataset:** 41 TCP/IP features, 25,192 training records, 5,039 validation records (20%). **BoT-IoT dataset:** Multi-class with DoS, DDoS, reconnaissance, and normal traffic (UNSW Cyber Range Lab). Train-test split: 80–20%. Includes various attack types and subcategories.

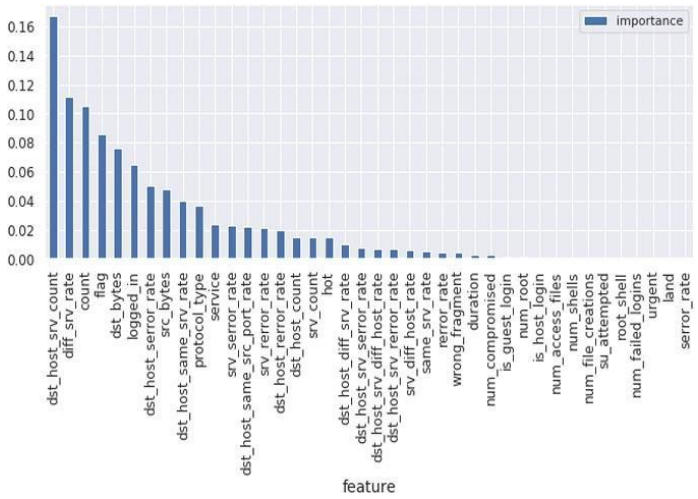


Figure 4: Features of the NID dataset

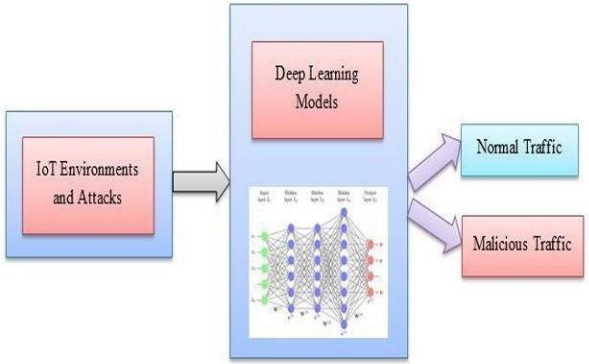


Figure 3. IDS using deep learning models

- **Traditional IDS limitations:** Not secure for IoT due to resource constraints. **High computational requirements:** Needs power and storage. **False positives:** Anomaly-based IDS prone to misclassification. **Dataset issues:** KDDCup99 lacks homogeneous IoT-specific data.
- **Accuracy:** 99.51% (NID), 95.55% (BoT-IoT). **Class-specific accuracy:** DDoS(86.18%), DoS (99.97%), Normal (100%), Reconnaissance (100%).**Confusion matrix analysis** and training/validation accuracy curvespresented.
- **IoT-specific security:** Need lightweight IDS for constrained devices. **Energy limitations:** IoT devices cannot run advanced IDS. **Unattended operation:** Vulnerability to malicious access and eavesdropping. **Dataset validity:** Need benchmark IoT-specific datasets for fair evaluation.

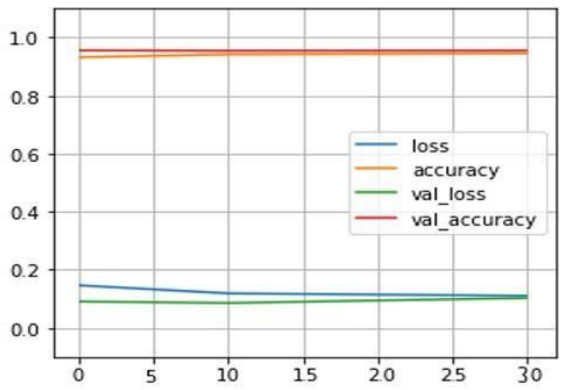
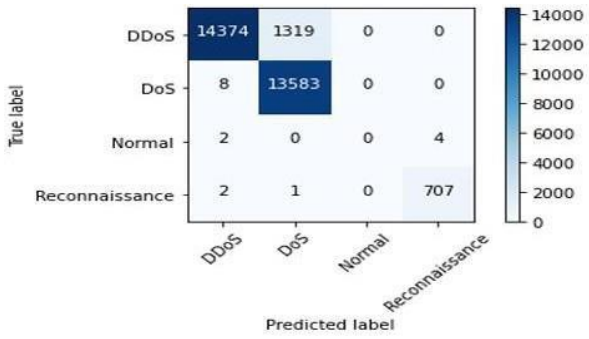


Figure 8: Training and validation accuracies on BoT-IoT dataset



LITERATURE REVIEW

3. Paper Title : Deep Learning in IoT Intrusion Detection

- **Survey-based methodology**: Systematic review of ~100 papers. **Classification by model types**: DNNs, CNNs, RNNs (LSTM/GRU), CNN-RNN hybrids, Boltzmann Machines, DBNs, Autoencoders, GANs. Literature search using Scopus with IoT, deep learning, and IDS criteria.
- **Multiple benchmark datasets**: NSL-KDD (148,517 samples), ISCX 2012 (500k+ samples), UNSW-NB15 (257,673 samples), CIDDS-001 (2M+ samples), CICIDS 2017 (618,976 samples). **Challenge**: Outdated datasets not reflecting modern IoT attacks.

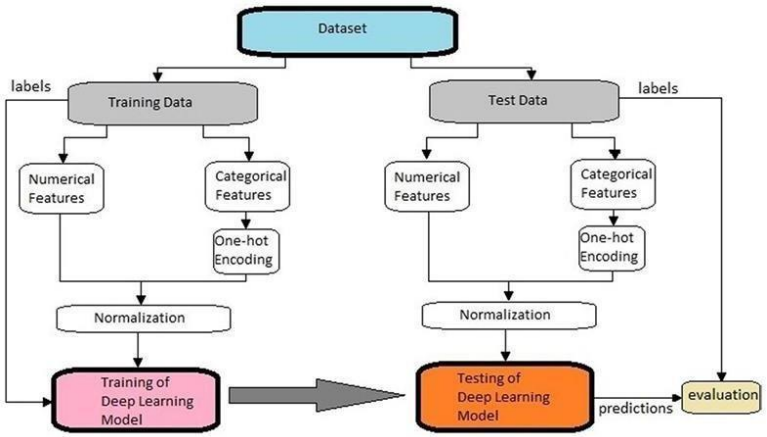


Fig. 5 Model training and testing

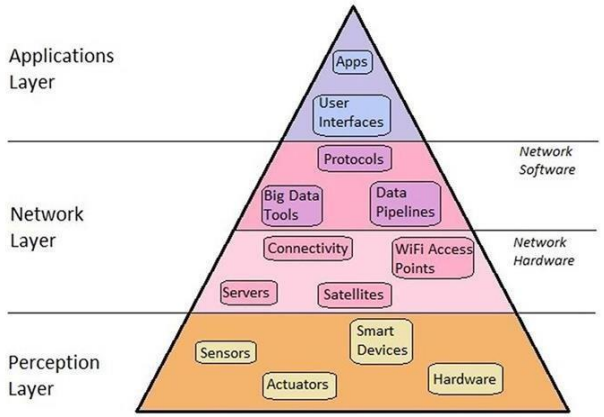
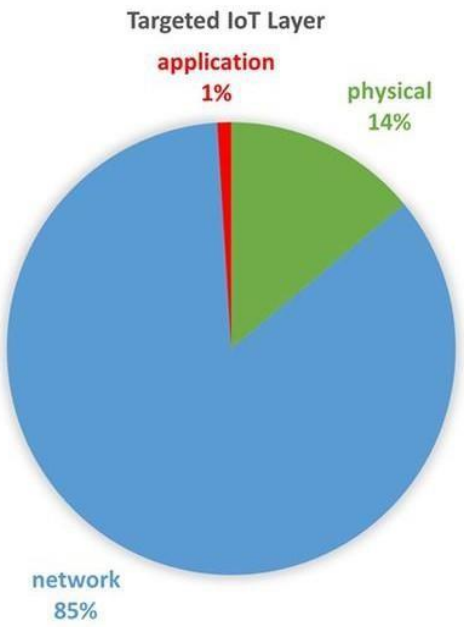
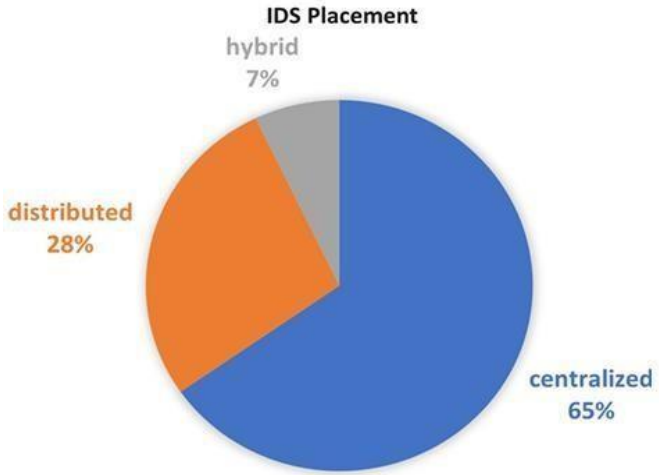


Fig. 2 IoT architecture

- **Accuracy ranges**: CNNs (99.13% ISCX2012), RNNs (99.99% ISCX2012), CNN-RNN hybrids (99.92%), DBNs (99.45% NSL-KDD), Autoencoders (99.96% NSL-KDD). **Additional metrics**: FPR, DR, Precision, Recall, F-measure reported.
- **Real-world validation**: Gap between simulated and actual IoT environments. **Standardization**: Lack of consistent benchmarks and evaluation metrics.
- **Scalability**: DL deployment is challenging on resource-constrained IoT devices.





LITERATURE REVIEW

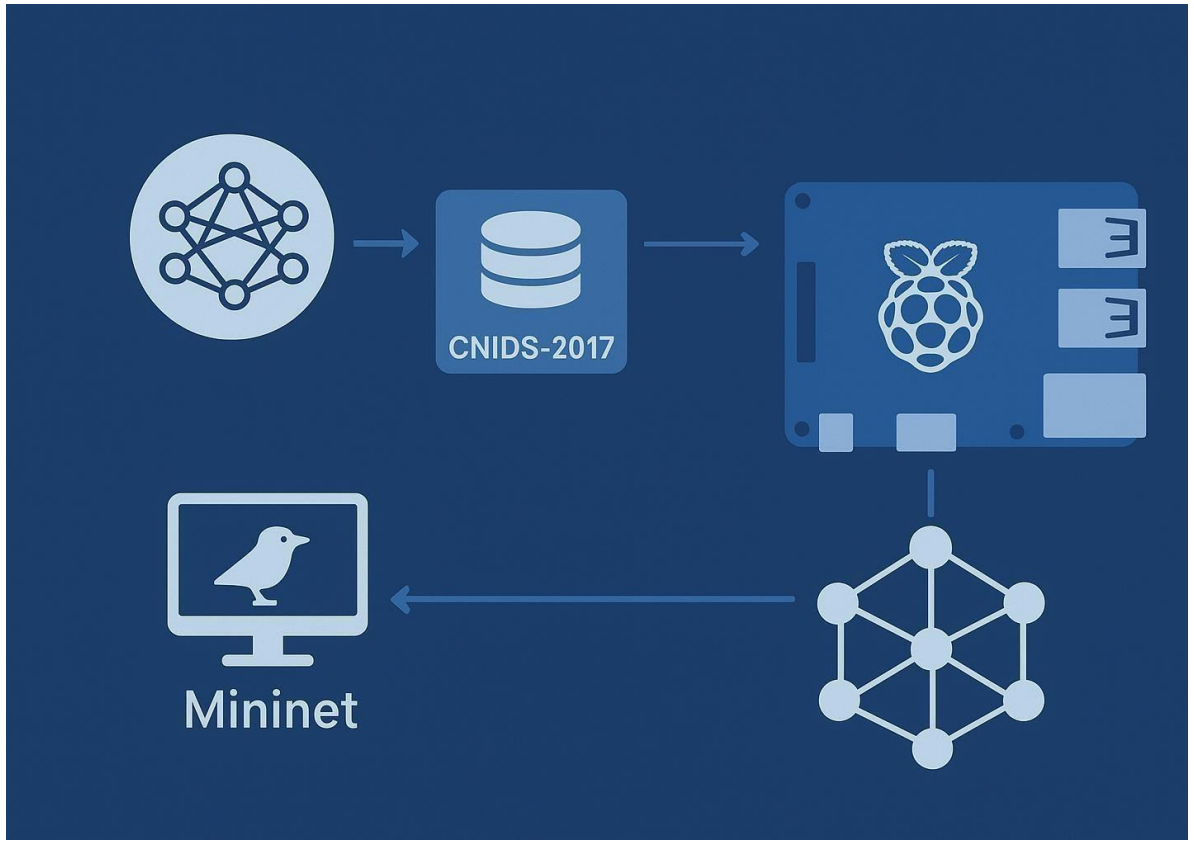
Paper Title	Methodology/ Implementation	Data set Description	Performance Parameter	Limitations/ Disadvtanges	Gaps Identified
4.Deep-IDS: A Real-Time Intrusion Detector for IoT Nodes Using Deep Learning	LSTM-based model, real-time blocking at 70% threshold, Raspberry Pi 4 (TensorFlow).	CIC-IDS2017, 56k cleaned records, 78 features, 15 classes.	Accuracy 97.67%, class-specific >93%.	Needs Pi 4 with 4GB RAM, threshold fixed, possible delays.	Limited dataset, real-time trade-off, deployment on diverse IoT devices.
5.Design and Implementation of a Machine Learning-Based Adaptive IDS on Raspberry Pi for Smart Home Network Security	Adaptive ML on Raspberry Pi, real-time logging, modular framework.	Simulated smart home data, DDoS, brute force, malware injection.	Accuracy 85%, response time 2s, low power use.	Limited Pi resources, performance drop with heavy loads.	Need real-world validation, scalability for larger networks.

PROBLEM STATEMENT

IoT devices in smart city networks are highly vulnerable to cyberattacks such as DoS, DDoS, and brute force. However, existing security systems are largely centralized and resource-intensive, making them unsuitable for providing efficient, real-time protection at the networkedge

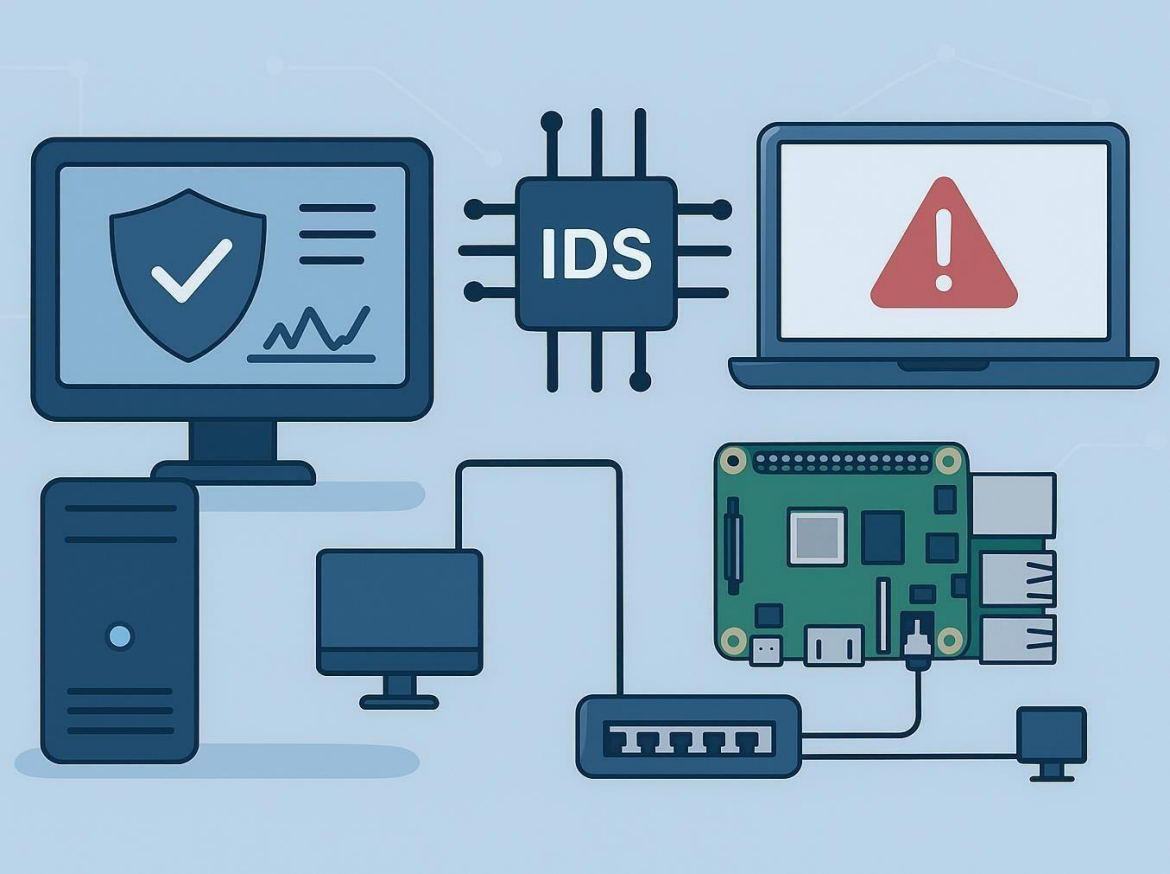
PROPOSED SOLUTION

- Train a **deep learning–based IDS model** (TSANet) using CICIDS-2017 dataset with 32 selected flow features.
- Use **TSANet (CNN + Attention + LSTM)** model for accurate and efficient intrusion detection.
- Export the trained model to **Torch Script** for efficient deployment.
- Deploy the model on a **Raspberry Pi** to act as an edge IDS.
- Use **Mininet** to simulate both benign and attack traffic (DoS, DDoS, BruteForce, Port Scan, Heartbleed, etc.).
- Real-time feature extraction and inference perform on the Pi.
- On detection of an attack → **blink an LED** on the RaspberryPi as an alert signal.



OBJECTIVES

- To design and train a deep learning–based IDS model using the CICIDS-2017dataset. ✅
- To convert the trained model into TorchScript for deployment on RaspberryPi. ✅
- To deploy and run the IDS model on Raspberry Pi for **real-time intrusion detection**. ✅
- To generate real time **traffic** Using Mininet for simulation. ✅



TOOL LEARNING

Machine Learning s Model Development

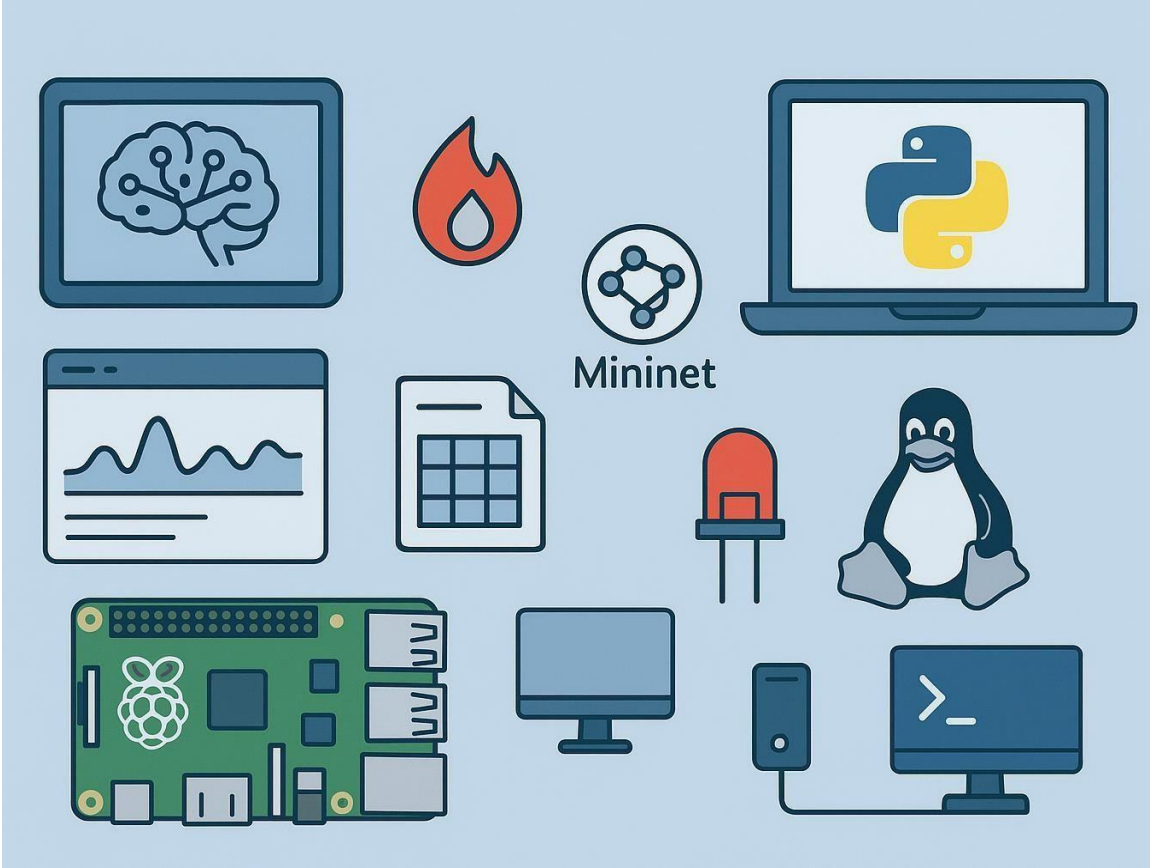
- PyTorch (model training, TorchScript export)
- Python (data preprocessing, feature extraction)

Networking s Traffic Generation

- Mininet (simulation of benign and attack traffic)
- CICIDS-2017 dataset (benchmark IDS dataset)

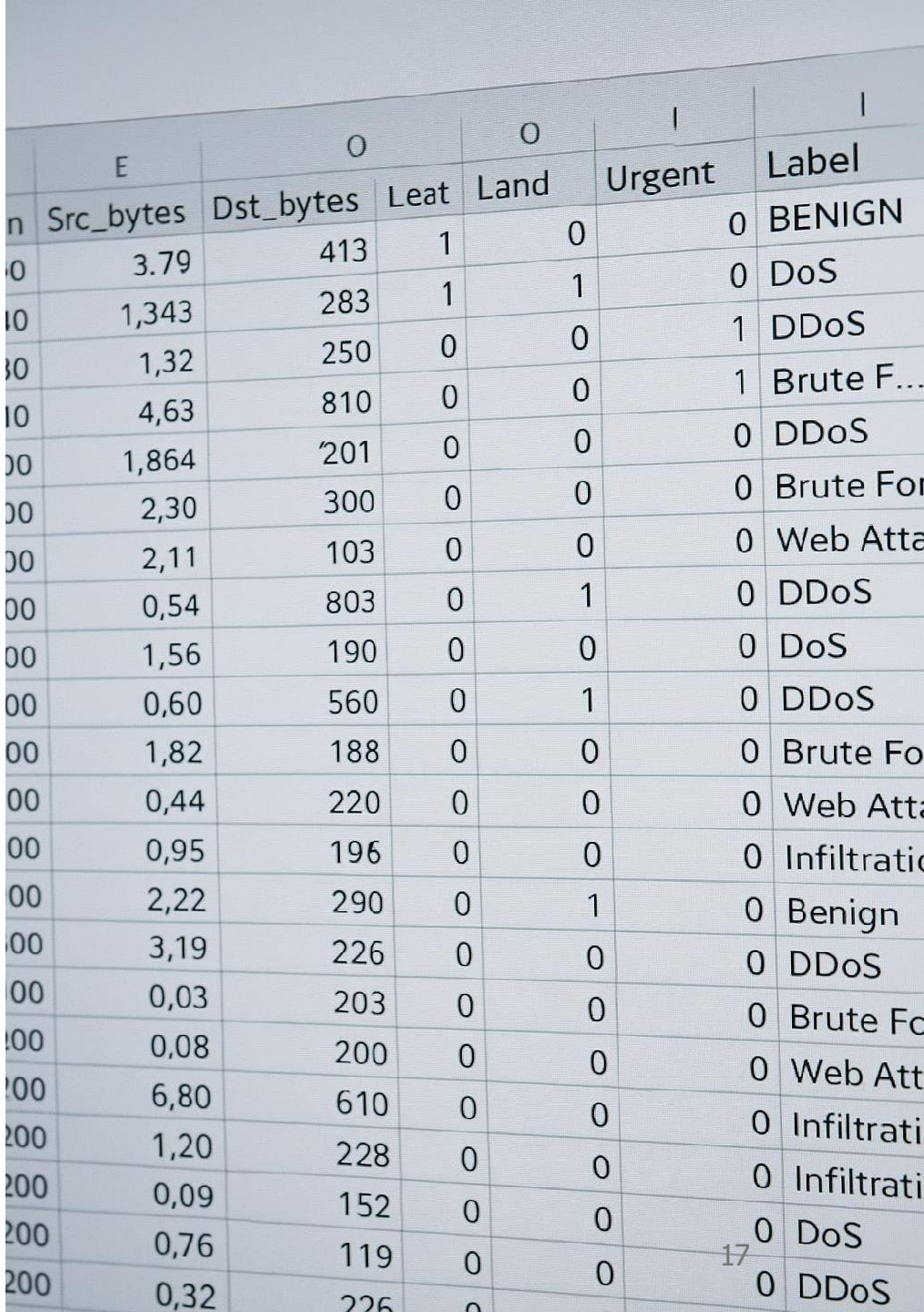
Deployment s Hardware

- Raspberry Pi (edge device for real-time IDS)
- GPIO control (LED alerts)
- Linux environment C SSH (device management)



DATASET DESCRIPTION

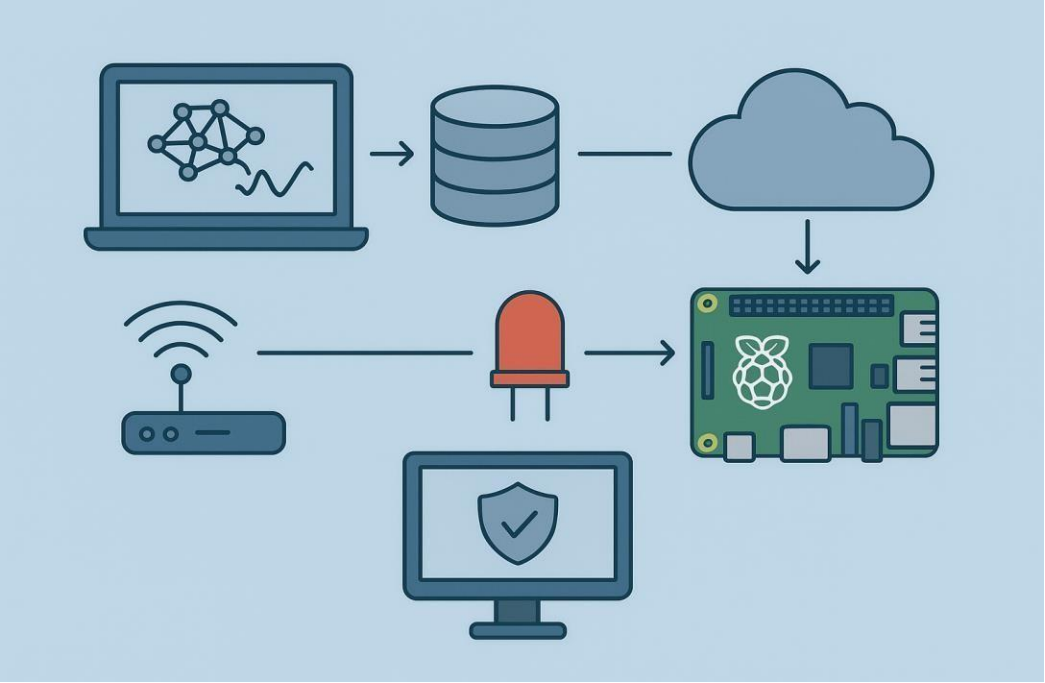
- The CICIDS2017 dataset is a benchmark dataset for intrusion detection, created by the Canadian Institute for Cybersecurity. It contains 8 CSV files, each representing traffic captured on different days and attack scenarios. Across all files, the dataset has 80 features (network flow characteristics such as packet counts, durations, byte rates, and flag counts) plus a Label column indicating *BENIGN* or specific attack types.
- Example file statistics:
- Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv – ~230,000 rows, 79 feature columns +Label
- Thursday-WorkingHours-Afternoon-Infiltration.pcap_ISCX.csv – ~170,000 rows, 78 feature columns +Label
- The dataset includes both benign traffic and multiple attack categories such as DoS, DDoS, Brute Force, Web Attacks, and Infiltration.

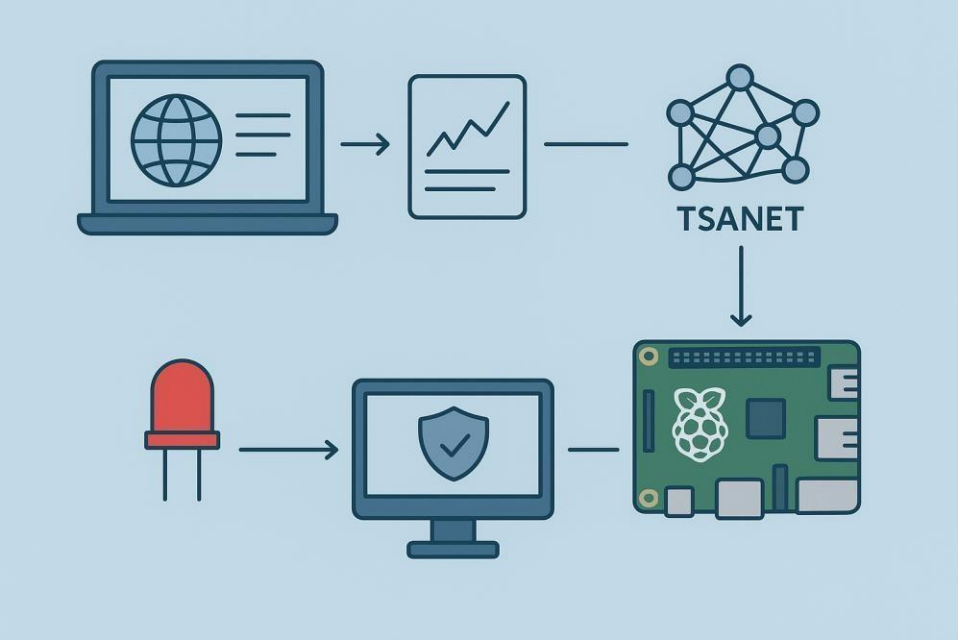


	E	O		O	I	I
n	Src_bytes	Dst_bytes	Leat	Land	Urgent	Label
0	3.79	413	1	0	0	BENIGN
0	1,343	283	1	1	0	DoS
0	1,32	250	0	0	1	DDoS
0	4,63	810	0	0	1	Brute F...
00	1,864	201	0	0	0	DDoS
00	2,30	300	0	0	0	Brute For
00	2,11	103	0	0	0	Web Atta
00	0,54	803	0	1	0	DDoS
00	1,56	190	0	0	0	DoS
00	0,60	560	0	1	0	DDoS
00	1,82	188	0	0	0	Brute Fo
00	0,44	220	0	0	0	Web Atta
00	0,95	196	0	0	0	Infiltratio
00	2,22	290	0	1	0	Benign
00	3,19	226	0	0	0	DDoS
00	0,03	203	0	0	0	Brute Fo
00	0,08	200	0	0	0	Web Att
00	6,80	610	0	0	0	Infiltrati
00	1,20	228	0	0	0	Infiltrati
00	0,09	152	0	0	0	DoS
00	0,76	119	0	0	0	DDoS
00	0,32	226	0			

EXPERIMENTALSETUP

- Environment
  Python-based setup with TensorFlow/PyTorch for model implementation and training.
- Data Source
  IoT security datasets such as CICIDS2017 for training and evaluation.
- Model Training C Testing
  TSANet (CNN + Attention + LSTM hybrid) trained to classify normal and attack traffic patterns.
- Traffic Simulation
  Network traffic generated using Mininet to simulate normal and malicious IoTactivities.
- Edge Deployment
  Trained model deployed on Raspberry Pi for lightweight, real-time execution.
- Real-Time Detection
  Raspberry Pi monitors incoming traffic and triggers LED alerts upon detecting attacks (DoS, DDoS, Heartbleed, etc.).





REQIUREMENTANALYSIS

In requirement analysis, we describe what the system should do and the conditions it should meet.

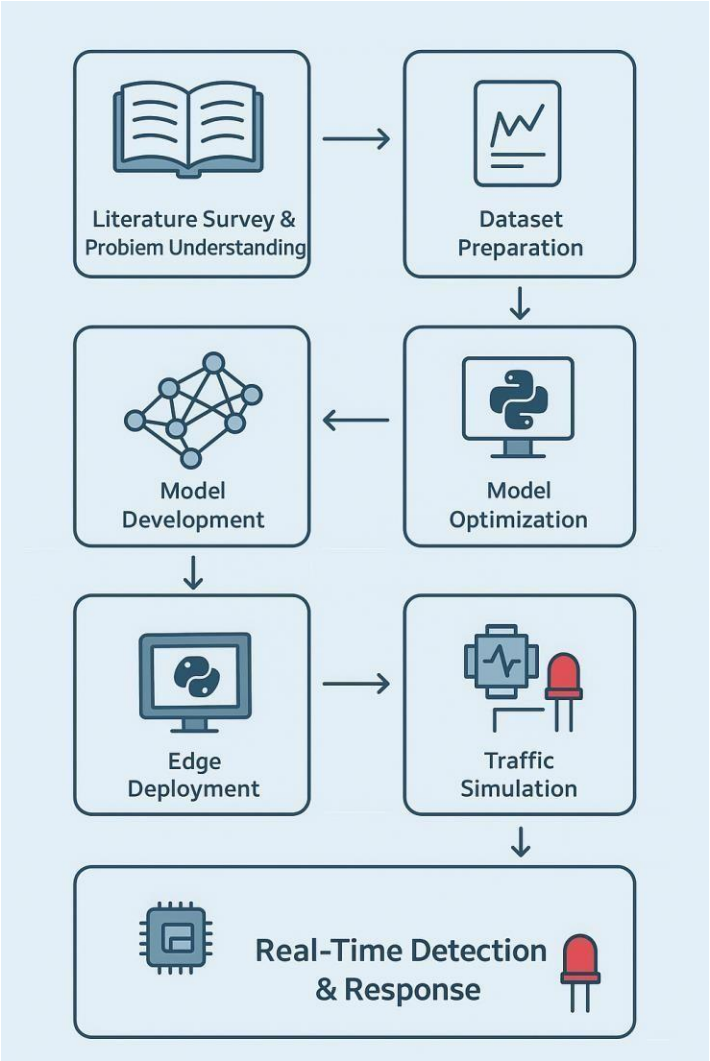
Functional Requirements

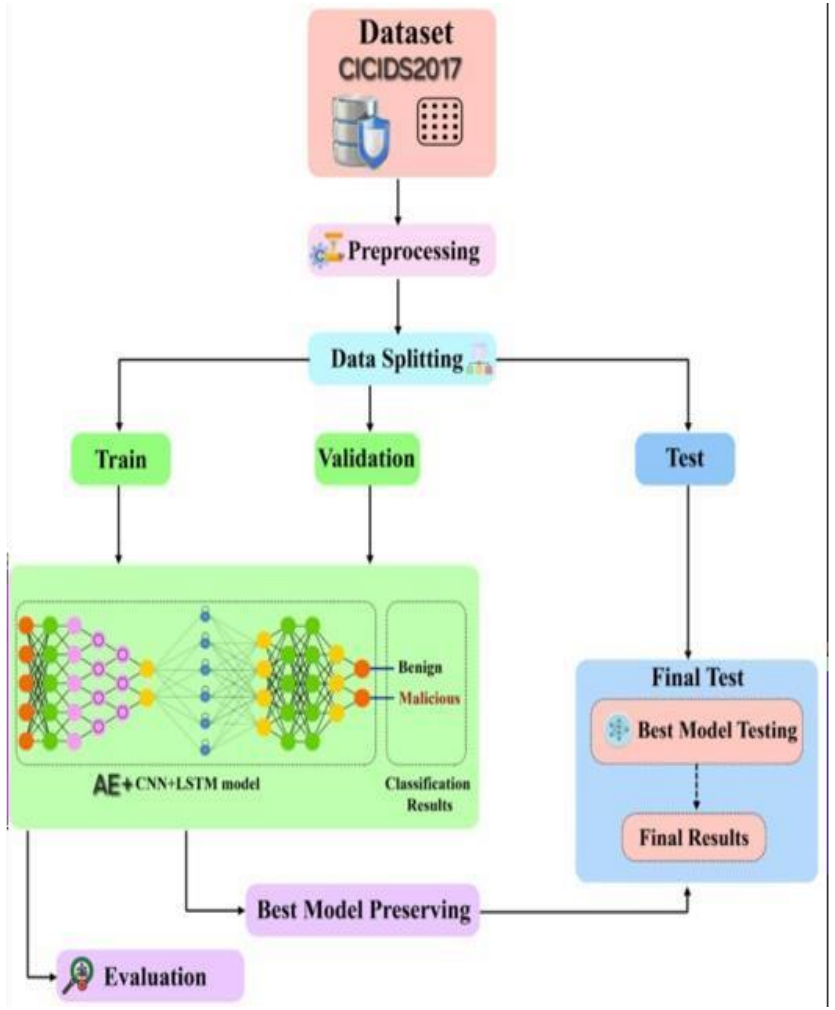
- Capture and process network traffic flows in real time.
- Extract 32 flow-level features from traffic.
- Classify traffic as *benign* or *attack* using TSANet model.
- Trigger LED alert on Raspberry Pi when attack is detected.

Non-Functional Requirements

- Real-Time Performance: Low-latency detection suitable for edge environments.
- Lightweight Deployment: Model optimized to run on Raspberry Pi.
- Scalability: System adaptable to different IoT attack types and traffic volumes.
- Usability: Easy integration with IoT/edge devices without complex setup.

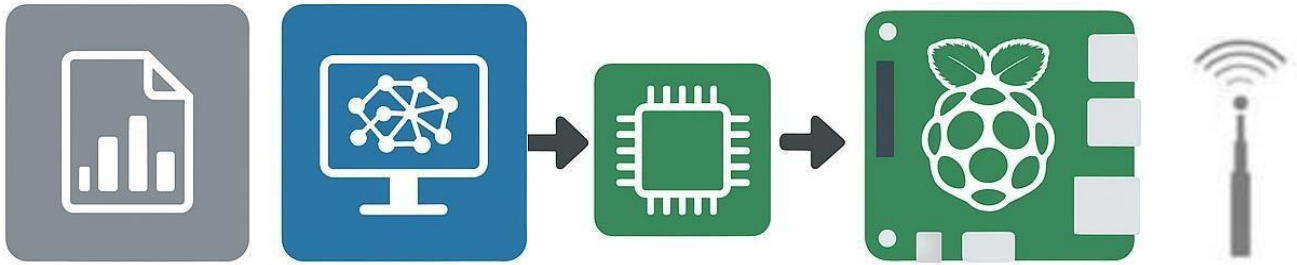
ROADMAP


Literature Survey & Probiem Understanding
Dataset Preparation
Model Development
Model Optimization
Edge Deployment
Traffic Simulation
Real-Time Detection & Response


Dataset
CICIDS2017
Preprocessing
Data Splitting
Train
Validation
Test
Benign
Malicious
AE+CNN+LSTM model
Classification Results
Final Test
Best Model Testing
Final Results
Best Model Preserving
Evaluation

IMPLEMENTATION

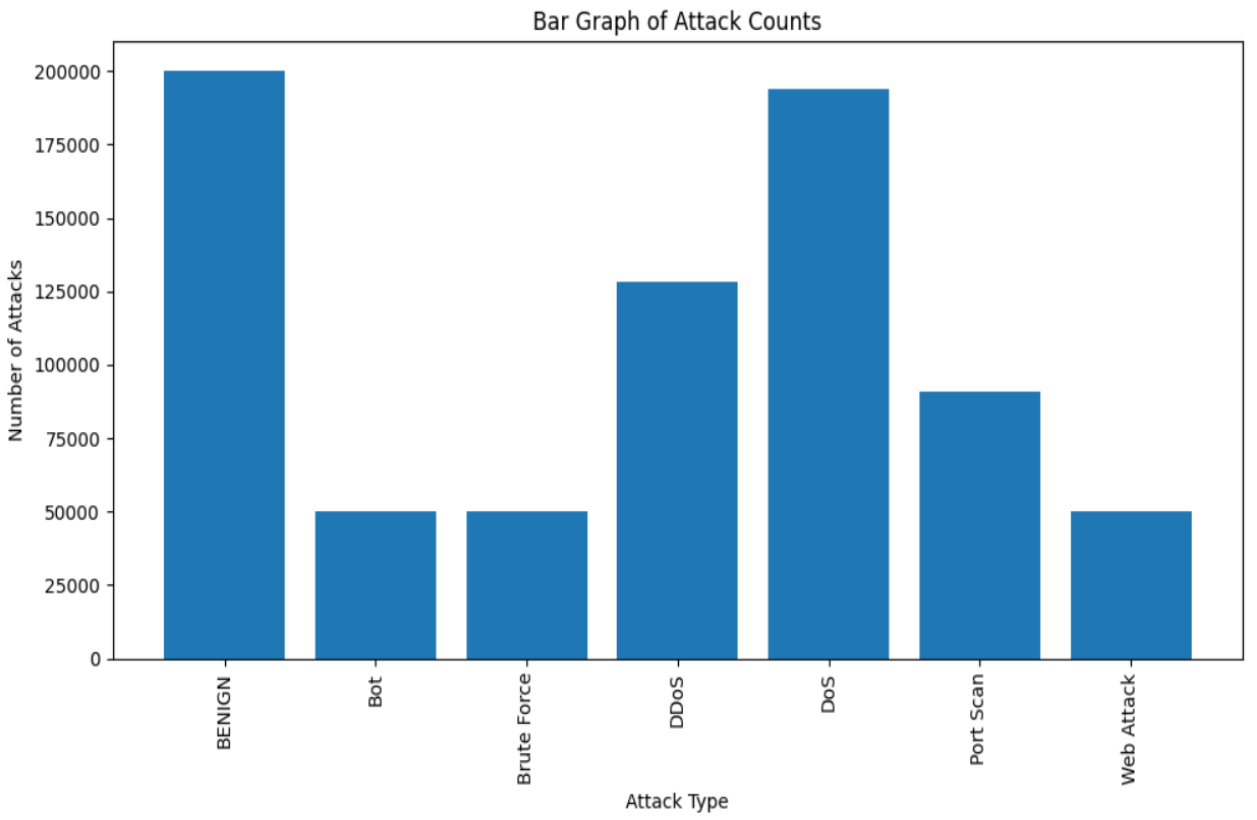
- **Dataset Preparation**
- **Model Design and Training**
- **Model Conversion for Deployment**
- **Deployment on Raspberry Pi**
- **Traffic Generation**



IMPLEMENTATION.

Dataset Preparation

- Collected and preprocessed the CICIDS-2017 dataset for intrusion detection tasks.
- Provides realistic, labeled network traffic for developing and testing Intrusion Detection Systems.
- Performed feature selection and normalization to prepare data for model training.
- Real users performed activities like web browsing, emails, chats, file transfers, video streaming, VoIP.
- Attack scenarios were simulated (DoS, DDoS, BruteForce, Botnet, SQL Injection, XSS, Infiltration, Heartbleed).
- Every record is tagged as either Benign or a Specific Attack Type.
- Makes it suitable for training deep learning models.



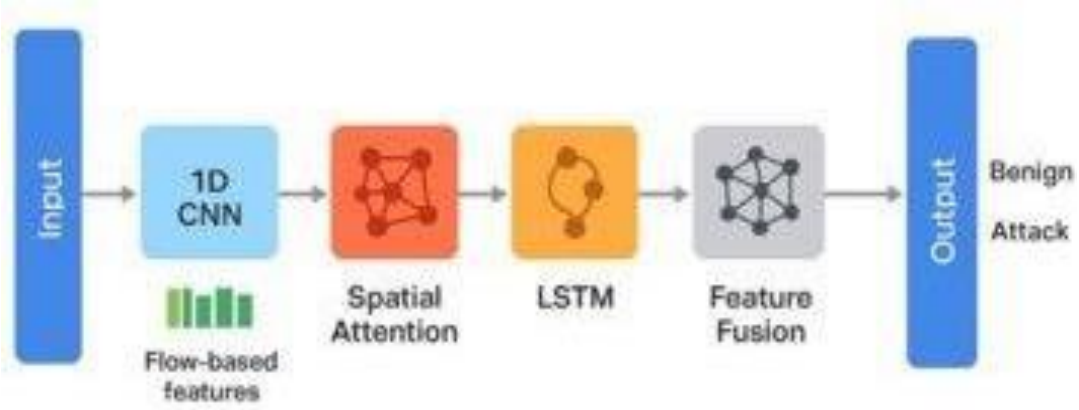
Dataset	Year	Key Features	Attack Coverage	Traffic Realism	Data Volume	Remarks / Justification
CICIDS-2017	2017	Rich flow-based features (80+), PCAPs available	Comprehensive: DoS, DDoS, Brute Force, Botnet, PortScan, Heartbleed, Web, Infiltration	High realism (real traffic patterns)	Large (~3 GB)	Chosen — well-structured, public, reproducible, and diverse attacks suitable for edge deployment
CSE-CIC-IDS-2018	2018	Updated traffic with additional botnet attacks	Slightly more IoT-like	High realism	Very large (~16 GB)	Good dataset but too large and preprocessing-heavy for edge-based model training
TON-IoT	2020	IoT + network telemetry	IoT-specific (8 attacks)	High realism	Small–Medium	Modern dataset but less standardized for flow-based deep learning
IoT-23	2021	Focused on IoT malware traffic	IoT-centric (malware, Mirai, Gafgyt)	Medium realism	Medium	Great for IoT malware but limited diversity (mainly botnets)

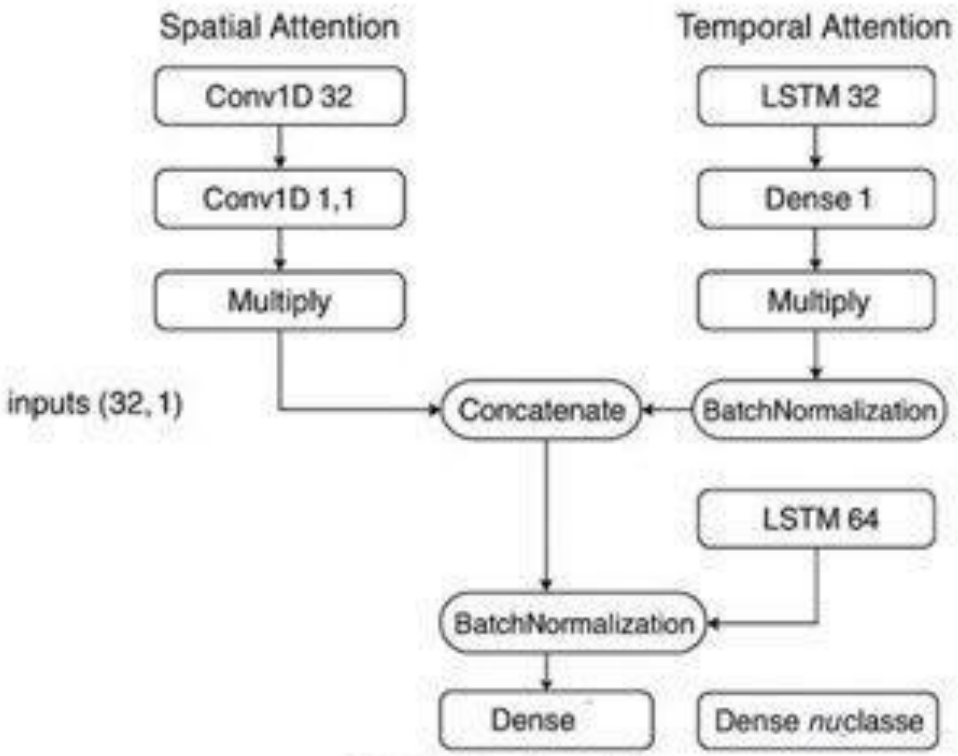
IMPLEMENTATION.

Model Design: TSANet

- A **hybrid deep learning model** designed for intrusion detection.
- Combines **CNN + Attention + LSTM** for capturing both spatial and temporal patterns in network traffic.
- **CNN** → extracts local/spatial patterns between features.
- **Attention** → focuses on the most relevant features and times.
- **LSTM** → models how traffic changes **over time**.

This combo is powerful for **intrusion detection** because attacks have both:
What features matter (spatial)
When they happen (temporal).





TSANet Architecture

Architecture Components

1. **Input Layer**:

Inputs (32,1)

32 = number of extracted flow features (e.g., packet count, size, duration, inter-arrival time).

1 = channel dimension (since each feature is a single scalar).

The model begins with **32 traffic flow features per sample**.

2. **Spatial Attention Path (Which features matter?)**:

Conv1D (32 filters): Learns local correlations among the 32 features.

Conv1D (1 filter, kernel size 1): Produces **1 attention score per feature**.

Multiply: Weights the original features with learned scores → important features are highlighted, unimportant ones suppressed.

Outcome: Spatially attended feature map.

3. **Temporal Attention Path (When do features matter?)**:

LSTM (32 units): Captures sequential/temporal dependencies in traffic patterns.

Dense (1): Converts hidden states into scalar attention scores.

Multiply: Applies weights to emphasize the most critical time steps.

Outcome: Temporally attended feature sequence

4. **Fusion Layer**:

Concatenate: Joins outputs from spatial + temporal attention.

Batch Normalization: Stabilizes training by normalizing values.

Outcome: Unified spatial–temporal feature representation.

5. **Final Representation**:

LSTM (64 units): Refines the fused representation in a higher-capacity hidden space.

(Here **64 = hidden state size**).

Dense Layer: Prepares features for classification.

Dense (n_classes): Outputs probabilities for each class (Benign, DoS, DDoS, Brute Force, etc.)

IMPLEMENTATION.

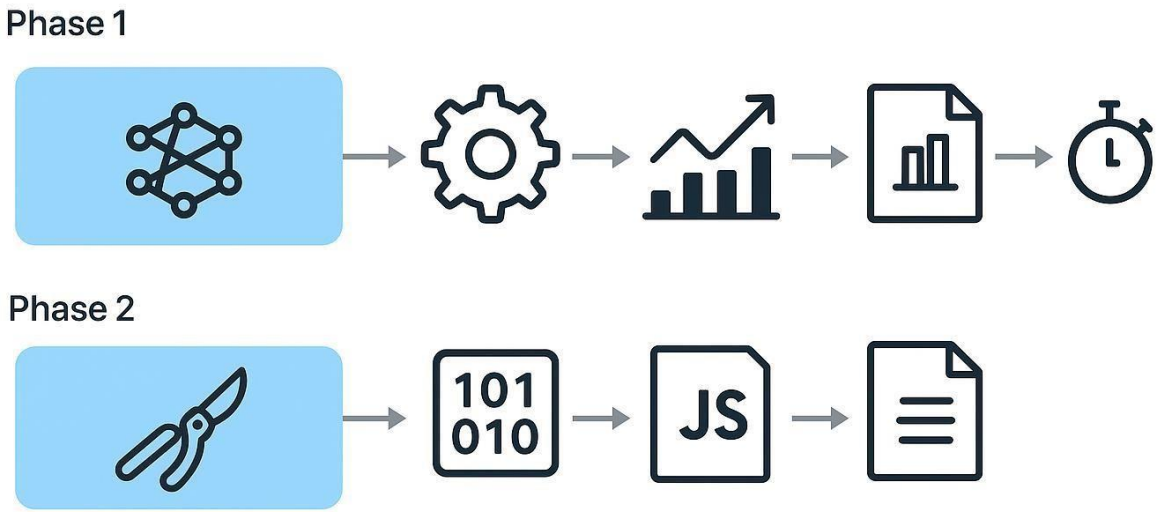
ModelTraining

Phase1:FullFeatureTraining

- **Dataset:**CIC-IDS2017(32features).
- **Learning Rate**=0.001.
- **BatchSize:**256,Epochs:30.
- **Loss Function:** Cross-Entropy Loss (multi-class classification).

Phase2:ModelOptimization

- **Pruning:**Removed redundant neurons→reduced Model size, improved inference speed.
- **Quantization-aware Training:** Prepared for low-precision operations(int8/float16).
- **TorchScript Conversion:** For lightweight deployment(onedgedeviceslikeRaspberryPi).

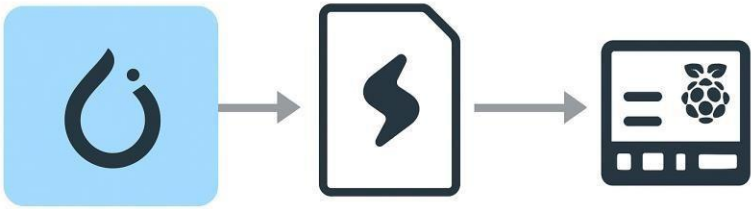


Algorithm	Why Tried	Results (Accuracy/Performance)	Limitations
Decision Tree	Simple, interpretable, works well on tabular data	~85% accuracy on CIC-IDS 2017	- Overfitting on training data- Poor generalization to unseen traffic- Cannot handle sequential/time-based patterns in attacks
SVM (Support Vector Machine)	Strong for binary classification, good for small datasets	~88% accuracy (binary), but drops significantly in multi-class intrusion detection	- Very high computational cost for large dataset- Slow training s inference- Poor scalability for real-time mobile deployment
TSANet (CNN + LSTM + Attention)	Captures both feature patterns time dependencies	67–68% accuracy (offline)63% accuracy	Optimized with pruning s quantization for IoT Devices (Resources Constrained Devices) →lightweight, fast (38ms/flow)

IMPLEMENTATION

Model Conversion for Deployment

- Converted the trained PyTorch model into **TorchScript** format.
- Optimized the model for deployment on **RaspberryPi** with limited resources.
- Raw PyTorch models are **too heavy for Raspberry Pi**
- Needed a **lightweight, portable format** to ensure **low-latency real-time detection**.
- Solution: **Optimize + Convert to TorchScript** for deployment.



IMPLEMENTATION.

Step 1: Model Pruning

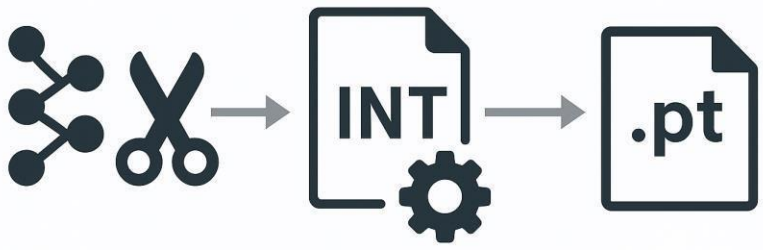
- Removed redundant neurons and weights.
- Reduced model size significantly.
- Improved inference speed by ~43%.
- Accuracy slightly improved (67.35% → 68.3%).

Step 2: Quantization-Aware Training (QAT)

- Trained the model with **lower precision (int8/float16)**.
- Reduced **memory footprint** and improved **execution speed**.
- Minimal accuracy loss, but real-time feasible on Raspberry Pi.

Step 3: TorchScript Conversion

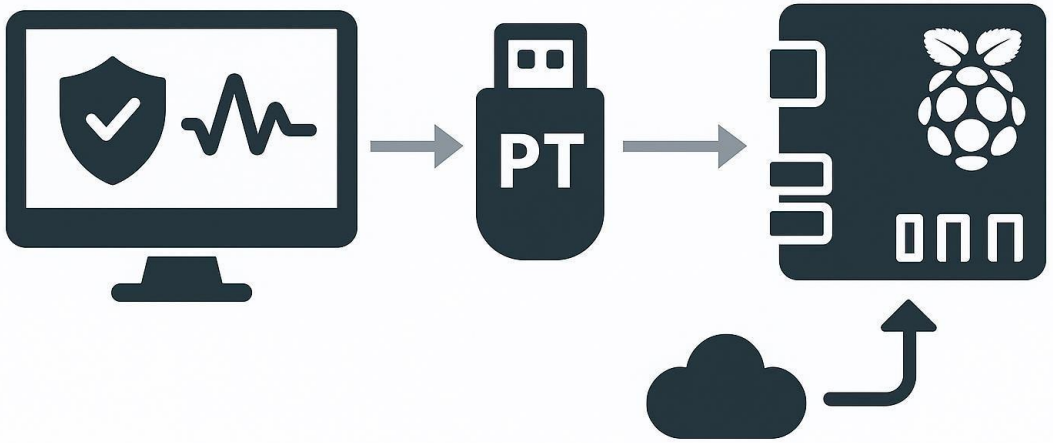
- Exported trained PyTorch model to **TorchScript (.pt)** format.
- TorchScript allows running the model **without the full PyTorch library**.
- Supports **just-in-time (JIT) compilation** for faster execution.



IMPLEMENTATION

Deployment on Raspberry Pi

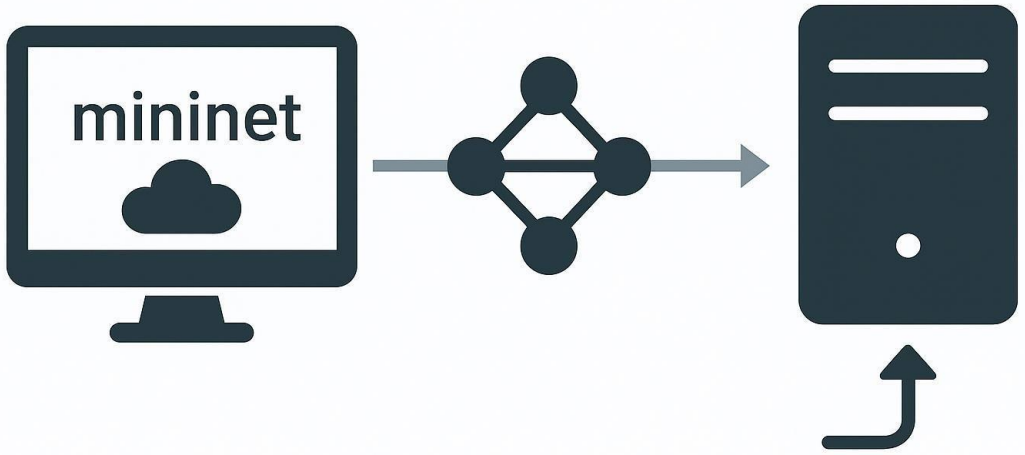
- Deployed the TorchScript IDS model on to the Raspberry Pi.
- Integrated real-time packet capturing for live intrusion detection.
- Model Transfer Trained and optimized TorchScript model (.pt file) was **transferred via USB (pendrive)** to Raspberry Pi.
- File placed in the working directory (/home/pi/IDS/
- Raspberry Pi Environment Setup Installed required dependencies: **Python 3.x** , PyTorch , **Scapy / Socket libraries** for traffic feature extraction. TorchVision C NumPy
- Whatever the network will Comes now From mininet Will bedetected By the Model



IMPLEMENTATION

Traffic Generation

- Plan to generate both **benign** and **Attack traffic** using **Mininet.**
- Will evaluate the IDS performance in real- time under controlled scenarios.



NOVELTY OF THE PROJECT

- **TSANet hybrid model (CNN + Attention + LSTM)** — a custom architecture designed for flow-based IDS that extracts spatial patterns and captures temporal behavior, improving detection of stealthy and multi-step attacks.
- **Edge-first deployment on Raspberry Pi** — while many studies run models on powerful servers or GPUs, this work demonstrates the model running on a low-cost Raspberry Pi, proving practical, real-world edge inference on resource-limited hardware.
- **Lightweight optimization for deployment** — the model is compressed and exported (TorchScript +quantization/optimizations) so it meets memory and speed limits of edge devices while keeping accuracy high.
- **Unique use of Mininet for traffic simulation** — instead of relying only on static datasets, realistic attack and normal traffic are generated in Mininet to test how the model performs in real-time conditions.

RESULTS

Full Feature Training

Result:

- Training Accuracy ≈ 98%
- Test Accuracy ≈ 97.35%

Model Optimization Result:

- Accuracy improved slightly to 98.3%
- Inference time reduced by ~44%.

```
"🚦 Starting real-time IDS... press Ctrl+C
"🚨 Prediction: BENIGN (conf=0.8604)\n"
"🚨 Prediction: BENIGN (conf=0.6664)\n"
"🚨 Prediction: Heartbleed (conf=0.4989)\n"
"🚨 Prediction: BENIGN (conf=0.9017)\n"
"🚨 Prediction: BENIGN (conf=0.8954)\n"
"🚨 Prediction: DoS (conf=0.4847)\n"
"🚨 Prediction: DoS (conf=0.7415)\n"
"🚨 Prediction: BENIGN (conf=0.8859)\n"
"🚨 Prediction: DoS (conf=0.5000)\n"
"🚨 Prediction: BENIGN (conf=0.9593)\n"
"🚨 Prediction: BENIGN (conf=0.9117)\n"
"🚨 Prediction: BENIGN (conf=0.9123)\n"
"🚨 Prediction: BENIGN (conf=0.5800)\n"
"🚨 Prediction: Heartbleed (conf=0.6001)\n"
"🚨 Prediction: Heartbleed (conf=0.4682)\n"
"🚨 Prediction: BENIGN (conf=0.5594)\n"
"🚨 Prediction: BENIGN (conf=0.8834)\n"
"🚨 Prediction: BENIGN (conf=0.8825)\n"
"🚨 Prediction: Heartbleed (conf=0.5531)\n"
"🚨 Prediction: BENIGN (conf=0.8912)\n"
"🚨 Prediction: BENIGN (conf=0.5482)\n"
"🚨 Prediction: BENIGN (conf=0.8099)\n"
"🚨 Prediction: BENIGN (conf=0.9201)\n"
"🚨 Prediction: Heartbleed (conf=0.7096)\n"
"🚨 Prediction: BENIGN (conf=0.8419)\n"
"🚨 Prediction: DoS (conf=0.7487)\n"
"🚨 Prediction: BENIGN (conf=0.8904)\n"
"🚨 Prediction: DoS (conf=0.4722)\n"
"🚨 Prediction: BENIGN (conf=0.8908)\n"
"🚨 Prediction: BENIGN (conf=0.9222)\n"
"🚨 Prediction: DoS (conf=0.7348)\n"
"🚨 Prediction: BENIGN (conf=0.5535)\n"
"🚨 Prediction: BENIGN (conf=0.9241)\n"
```

Results

```
🚦 Starting real-time IDS... press Ctrl+C to stop.
🚨 Prediction: BENIGN (conf=0.8604)
🚨 Prediction: BENIGN (conf=0.6664)
🚨 Prediction: Heartbleed (conf=0.4989)
🚨 Prediction: BENIGN (conf=0.9017)
🚨 Prediction: BENIGN (conf=0.8954)
🚨 Prediction: DoS (conf=0.4847)
🚨 Prediction: DoS (conf=0.7415)
🚨 Prediction: BENIGN (conf=0.8859)
🚨 Prediction: DoS (conf=0.5000)
🚨 Prediction: BENIGN (conf=0.9593)
🚨 Prediction: BENIGN (conf=0.9117)
🚨 Prediction: BENIGN (conf=0.9123)
🚨 Prediction: BENIGN (conf=0.5800)
🚨 Prediction: Heartbleed (conf=0.6001)
🚨 Prediction: Heartbleed (conf=0.4682)
🚨 Prediction: BENIGN (conf=0.5594)
🚨 Prediction: BENIGN (conf=0.8834)
🚨 Prediction: BENIGN (conf=0.8825)
🚨 Prediction: Heartbleed (conf=0.5531)
🚨 Prediction: BENIGN (conf=0.8912)
🚨 Prediction: BENIGN (conf=0.5482)
🚨 Prediction: BENIGN (conf=0.8099)
🚨 Prediction: BENIGN (conf=0.9201)
🚨 Prediction: Heartbleed (conf=0.7096)
...
```

RESULTS AFTER DEPLOYMENT ON EDGE DEVICE

REFERENCES

8 R. Morshedi, S. M. Matinkhah, and M. T. Sadeghi, "Intrusion Detection for IoT NetworkSecurity with Deep Learning," *Journal of Artificial Intelligence and Data Mining*, vol. 12,no. 1, pp. 37–55, 2024, doi: 10.22044/jadm.2023.13539.2471.

9 P. Spadaccino and F. Cuomo, "Intrusion Detection Systems for IoT: Opportunities andChallenges Offered by Edge Computing and Machine Learning," Sapienza Univ. ofRome, Italy, Conf. Paper, 2021.

10 A. Fatani, M. Abd Elaziz, A. Dahou, M. A. A. Al-Qaness, and S. Lu, "IoT IntrusionDetection System Using Deep Learning and Enhanced Transient Search Optimization,"*IEEE Access*, vol. 9, pp. 123448–123463, Sep. 2021, doi:10.1109/ACCESS.2021.3109081.

11 D. Gupta, N. Yadav, S. Pande, and A. Khamparia, "Intrusion Detection System on IoT with 5G Network Using Deep Learning," *Wireless Communications and Mobile Computing*, vol. 2022, Article ID 9304689, pp. 1–13, Mar. 2022, doi: 10.1155/2022/9304689.

12 A. Jimenez, "Using Machine Learning for Raspberry Pi Network Intrusion Detection," M.S. thesis, Dept. Comput. Sci., California State Polytechnic Univ., Pomona, USA, 2021.

13 B. F. Kayode, A. S. Akintola, O. Fagbohun, E. Anaesiuba, O. Ojumah, O. Odimayo, … and P. C. Amajuoyi, "Temporal-Spatial Attention Network (TSAN) for DoS Attack Detection in Network Traffic," *arXiv preprint*, arXiv:2503.16047, 2025.

14 R. Dahiya, M. Sharma, and M. Kumar, "Raspberry Pi IDS: A Fruitful Intrusion Detection System for IoT," in *Proc. 2nd Int. Conf. Innovative Practices in Technology and Management (ICIPTM)*, Noida, India, Mar. 2024, pp. 1–6, doi: 10.1109/ICIPTM60116.2024.10572179.

15 O. O. Olakanmi and S. O. Olatunji, "Optimized Ensemble Deep Learning for Real-Time Intrusion Detection on Resource-Constrained Raspberry Pi Devices," *Procedia Computer Science*, vol. 231, pp. 1991–1998, 2024, doi: 10.1016/j.procs.2024.03.245.

REFERENCES

1 I. Idrissi, M. Azizi, and O. Moussaoui, "A Lightweight Optimized Deep Learning-based Host-Intrusion Detection System Deployed on the Edge for IoT," *International Journal of Computing and Digital Systems*, vol. 11, no. 1, pp. 210–220, Jan. 2022, doi: 10.12785/ijcds/110117.

2 T. Saba, A. Rehman, T. Sadad, H. Kolivand, and S. A. Bahaj, "Anomaly-based Intrusion Detection System for IoT Networks through Deep Learning Model," *Computers & Electrical Engineering*, vol. 99, p. 107818, 2022, doi: 10.1016/j.compeleceng.2022.107818.

3 S. Tsimenidis, T. Lagkas, and K. Rantos, "Deep Learning in IoT Intrusion Detection," *Journal of Network and Systems Management*, vol. 30, no. 8, pp. 1–40, 2022, doi: 10.1007/s10922-021-09621-9.

4 S. Racherla, P. Sripathi, N. Faruqui, M. A. Kabir, M. Whaiduzzaman, and S. A. Shah, "Deep-IDS: A Real-Time Intrusion Detector for IoT Nodes Using Deep Learning," *IEEE Access*, vol. 12, pp. 63584–63598, May 2024, doi: 10.1109/ACCESS.2024.3396461.

5 R. Adrian, R. D. Mandasari, and S. Alam, "Design and Implementation of a Machine Learning-Based Adaptive IDS on Raspberry Pi for Smart Home Network Security," *SITEKIN: Jurnal Sains, Teknologi dan Industri*, vol. 22, no. 2, pp. 334–343, Jun. 2025.

6 Y. Otoum and A. Nayak, "DL-IDS: A Deep Learning-Based IntrusionDetectionFramework for Securing IoT," *Transactions on EmergingTelecommunicationsTechnologies*, vol. 33, no. 3, p. e3803, Mar. 2022, doi:10.1002/ett.3803.

7 A. Gueriani, H. Kheddar, and A. C. Mazari, "Enhancing IoT Security with CNN andLSTM-Based Intrusion Detection Systems," Univ. of Medea, Algeria, Tech. Rep.,2023.

THANK YOU